

Упражнение 3: Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

1. зад. Да се напише програма, която прилага цикъл `while` и извежда всички положителни цели числа по-малки от 5, при зададена начална стойност 1, след прекратяване на цикъла да се изведе съобщение: "Програмата е прекратена".

Примерно решение:

```
number = 1
while number < 5:
    print(f"number = {number}")
    number += 1
print("Програмата е прекратена")
```

2. зад. Да се допълни задача 1 с изключение, при което се извежда съобщение: "Цикълът приключи".

Примерно решение:

```
number = 1
while number < 5:
    print(f"number = {number}")
    number += 1
else:
    print("Цикълът приключи")
print("Програмата е прекратена")
```

3. зад. Да се преработи задача 2 така, че началната стойност да е 10.

Примерно решение:

```
number = 10
while number < 5:
    print(f"number = {number}")
    number += 1
else:
    print(f"number = {number}. Цикълът завърши")
print("Програмата е прекратена")
```

4. зад. Като се приложи цикъл `for`, да се изведе посимволно съдържанието на низа `Hello`.

Примерно решение:

```
message = "Hello"
for c in message:
    print(c)
```

5. зад. Да се допълни условието от зад.4 с допълнителен блок `else`, който се изпълнява след края на цикъла.

Примерно решение:

```
message = "Hello"
for c in message:
    print(c)
else:
    print(f"Последен символ: {c}. Цикълът завършва");
print("Програмата приключи")
```

6. зад. Да се създаде програма, чийто изход е таблицата за умножение. Задачата да се реализира с два вложени цикъла while.

7. зад. Да се създаде програма, чийто изход е комбинации от знаци a и b. Задачата да се реализира с два вложени цикъла for. Входните данни са низ "ab" и низ "ba".

8. зад. Да се създаде програма, която извежда всички числа по-малки от 5 чрез цикъл while. При проверка за стойност равна на 3, програмата се прекратява. Да се приложи операторът break.

9. зад. Да се създаде програма, която извежда всички числа по-малки от 5 чрез цикъл while. При проверка за стойност равна на 3, програмата прескача към следващата итерация. Да се приложи операторът continue.

10. зад. Да се дефинира функция say_hello(), без параметри, която съдържа един единствен израз, който отпечатва низа "Hello" в конзолата.

Примерно решение:

```
def say_hello():  
    print("Hello")
```

11. зад. Да се допълни задача 10, с допълнителен ред за принтиране на съобщение „Bye“, това съобщение, обаче не трябва да принадлежи на функцията.

Примерно решение:

```
def say_hello():  
    print("Hello")  
print("Bye")
```

12. зад. Да се промени задача 10, като се извърши трикратно извикване на функцията say_hello(), без параметър.

Примерно решение:

```
def say_hello():  
    print("Hello")  
say_hello()  
say_hello()  
say_hello()
```

Забележка: Ако функцията има един израз, тогава тя може да бъде поставена на същия ред като останалата част от дефиницията на функцията

```
def say_hello(): print("Hello")  
say_hello()
```

13. зад. Да се създадат и извикат функциите say_hello() и say_goodbye(). Първата да извежда съобщение: "Hello", втората "Good Bye".

Примерно решение:

```
def say_hello():  
    print("Hello")  
def say_goodbye():  
    print("Good Bye")  
say_hello()
```

say_goodbye()

14. зад. Да се дефинира функцията print_messages(), която има 2 локални функции: say_hello() и say_goodbye(). Първата да извежда съобщение: "Hello", втората съобщение "Good Bye". Да се изведе резултата от прилагане на функцията print_messages() и от локалната функция say_hello(). Опишете резултата от изпълнението.

Примерно решение:

```
def print_messages():
    # дефиниция на локални функции
    def say_hello(): print("Hello")
    def say_goodbye(): print("Good Bye")
    # извикване на локални функции
    say_hello()
    say_goodbye()
# Извикване на функцията print_messages
print_messages()
# Извън функцията print_messages say_hello не е налична
say_hello()
```

Забележка: Много функции могат да бъдат дефинирани в една програма. И за да се организират всички, един от начините да се организират е да се добави специална функция (обикновено наричана main).

Примерно преработено решение на задача 14:

```
def main():
    say_hello()
    say_goodbye()
def say_hello():
    print("Hello")
def say_goodbye():
    print("Good Bye")
# Извикване на основната функция
main()
```

15. зад. Да се дефинира функцията say_hello, с параметър name. Да се отпечата стойността на този параметър в конзолата с функцията за печат.

Примерно решение:

```
def say_hello(name):
    print(f"Hello, {name}")
say_hello("Tom")
say_hello("Bob")
say_hello("Alice")
```

16. зад. Да се допълни условието от задача 15, като се добави още един параметър age. Да се извика функцията и се отпечата резултата чрез функцията за печат.

Примерно решение:

```
print_person(name, age):
    print(f"Name: {name}")
    print(f"Age: {age}")
print_person("Tom", 37)
```

17. зад. Да се дефинира функцията say_hello(), да се добави параметър на функцията - name, който да е незадължителен, като се посочи стойност по подразбиране (стойност по подразбиране се извършва още в дефинирането на функцията).

Примерно решение:

```
def say_hello(name="Tom"):
    print(f"Hello, {name}")
say_hello()          # параметър name приема стойност "Tom"
say_hello("Bob")     # name = "Bob"
```

Забележка: Ако функцията има множество параметри, незадължителните параметри трябва да са след задължителните. При извикване на функцията, стойностите се предават като параметри на функцията по позиция.

18. зад. Да се допълни условието от задача 17 с втори параметър - age, като се зададе стойност по подразбиране на вторият параметър.

Примерно решение:

```
def print_person(name, age = 18):
    print(f"Name: {name} Age: {age}")
print_person("Bob")
print_person("Tom", 37)
```

19. зад. Да се преработи задача 18, така че да се предават стойности на параметри по име, а не по-позиция (предаването на стойности на параметри по име става, като при извикване на функцията се посочва името на параметъра и се присвоява съответната стойност).

Примерно решение:

```
def print_person(name, age):
    print(f"Name: {name} Age: {age}")
print_person(age = 22, name = "Tom")
```

Забележка: Знакът * позволява да се зададат кои параметри ще бъдат именувани – т.е. параметри, на които могат да се предават стойности само по име. Всички параметри, които се намират **вдясно от символа ***, получават **стойности само по име**:

Примерно решение:

```
def print_person(name, *, age, company):
    print(f"Name: {name} Age: {age} Company: {company}")
print_person("Bob", age = 41, company = "Microsoft")
# Name: Bob Age: 41 company: Microsoft
```

В този случай възрастта и параметрите на фирмата са именувани. Могат да бъдат именувани всички параметри, като се постави префикс към списъка с параметри с *:

Примерно решение:

```
def print_person(*, name, age, company):
    print(f"Name: {name} Age: {age} Company: {company}")
```

Ако, обаче, трябва да се дефинират параметри, на които могат да се предават стойности само по позиция, т.е. позиционни параметри, тогава може да се използва символа / : **всички параметри, които вървят преди символа / са позиционни** и могат да **получават само стойности по позиция**:

Примерно решение:

```
def print_person(name, /, age, company="Microsoft"):
    print(f"Name: {name} Age: {age} Company: {company}")
print_person("Tom", company="JetBrains", age = 24)
# Name: Tom Age: 24 company: JetBrains
print_person("Bob", 41)
# Name: Bob Age: 41 company: Microsoft
```

В този случай параметърът `name` е позиционен. За една функция може да се дефинират едновременно позиционни и именувани параметри.

Примерно решение:

```
def print_person(name, /, age = 18, *, company):
    print(f"Name: {name} Age: {age} Company: {company}")
print_person("Sam", company="Google") # Name: Sam Age: 18 company: Google
print_person("Tom", 37, company="JetBrains") # Name: Tom Age: 37 company: JetBrains
print_person("Bob", company="Microsoft", age = 42) # Name: Bob Age: 42 company: Microsoft
```

В този случай параметърът `name` се намира отляво на символа `/`, следователно е позиционен и задължителен и може да се предава само стойност по позиция. Параметърът `company` е именуван, защото се намира вдясно от символа `*`. Параметърът `age` може да получи стойност по име и по позиция.

Символът звездичка може да се използва за дефиниране на параметър, през който могат да се предават неопределен брой стойности. Това може да бъде полезно, когато трябва функция да получава множество стойности, но не е уточнен техният брой. Например, нека се дефинира функция за изчисляване на сумата от числа:

```
def sum(*numbers):
    result = 0
    for n in numbers:
        result += n
    print(f"sum = {result}")
sum(1, 2, 3, 4, 5) # sum = 15
sum(3, 4, 5, 6) # sum = 18
```

В този случай функцията за сумиране приема един параметър т.е. `*numbers`, но звездичката пред името на параметъра показва, че всъщност може да се предаде неопределен брой стойности или набор от стойности на мястото на този параметър.

20 зад. Нека се дефинира елементарна функция, която връща стойност.

Примерно решение:

```
def get_message():
    return "Hello abv.bg"
```

21 зад. Да се допълни условието от задача 20, като резултатът от тази функция да се присвои на променлива или да се използва като нормална стойност:

Примерно решение:

```
def get_message():
    return "Hello abv.bg"
message = get_message() # получаване на резултата на функции get_message в променливата message
print(message) # Hello abv.bg
# можете директно да предадете резултата от функцията get_message
print(get_message()) # Hello abv.bg
```

22. зад. Да се създаде функция double с един параметър number, която връща удвоеният резултат от параметъра. Да се изведе резултата в конзолата.

Забележка: Операторът return не само връща стойност, но и излиза от функцията. Следователно, той трябва да бъде дефиниран след останалите инструкции.

Примерно решение:

```
def get_message():  
    return "Hello abv.bg "  
    print("End of the function")  
print(get_message())
```

Примерно решение:

```
def print_person(name, age):  
    if age > 120 or age < 1:  
        print("Invalid age")  
    return  
    print(f"Name: {name} Age: {age}")  
print_person("Tom", 22)  
print_person("Bob", -102)
```

Забележка: В Python функцията всъщност представлява един тип. Така че може да се присвои функция на променлива и след това да се извика тази функция с помощта на променливата.

Примерно решение:

```
def say_hello(): print("Hello")  
def say_goodbye(): print("Good Bye")  
message = say_hello  
message()          # Hello  
message = say_goodbye  
message()          # Goog Bye  
message
```

По същия начин може да се извика функция с параметри чрез променлива и да се върне нейния резултат:

Примерно решение:

```
def sum(a, b): return a + b  
def multiply(a, b): return a * b  
operation = sum  
result = operation(5, 6)  
print(result)          # 11  
operation = multiply  
print(operation(5, 6))  # 30
```

Тъй като функцията в Python може да представлява същата стойност като низ или число, може съответно да се предаде като параметър на друга функция.

23. зад. Нека се дефинира функция, която отпечатва резултата от някаква операция (в случая тя е действие събиране) в конзолата

Примерно решение:

```
def do_operation(a, b, operation):
```

```

    result = operation(a, b)
    print(f"result = {result}")
def sum(a, b): return a + b
def multiply(a, b): return a * b
do_operation(5, 4, sum)          # result = 9
do_operation(5, 4, multiply)

```

Забележка: Една функция в Python може да върне друга функция. Например, ако се дефинира функция, която според стойността на параметъра връща една или друга операция:

Примерно решение:

```

def sum(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def select_operation(choice):
    if choice == 1:
        return sum
    elif choice == 2:
        return subtract
    else:
        return multiply
operation = select_operation(1)    # operation = sum
print(operation(10, 6))           # 16
operation = select_operation(2)    # operation = subtract
print(operation(10, 6))           # 4
operation = select_operation(3)    # operation = multiply
print(operation(10, 6))           # 60

```

Забележка: Ламбда изразите в Python са малки анонимни функции, които се дефинират с помощта на ламбда оператора.

24. зад. Нека се дефинира най-елементарен ламбда израз.

Примерно решение:

```

message = lambda: print("hello")
message()    # hello

```

Да се реализира съответствие с обикновена функция:

```

def message():
    print("hello")

```

Ако ламбда изразът има параметри, дефиницията става чрез ключовата дума lambda. Ако ламбда изразът върне резултат, тогава той се посочва след двоеточие.

25. зад. Нека се дефинира ламбда израз, който връща квадрата на число.

Примерно решение:

```

square = lambda n: n * n
print(square(4))    #16
print (square(5))   #25

```

Да се реализира съответствие с обикновена функция:

```

def square2(n): return n * n
print (square2(5))

```

Пример за подаване на ламбда израз като параметър:

```
def do_operation(a, b, operation):
    result = operation(a, b)
    print(f"result = {result}")
do_operation(5, 4, lambda a, b: a + b)    # result = 9
do_operation(5, 4, lambda a, b: a * b)
```

Връщане на ламбда изрази от функции:

```
def select_operation(choice):
    if choice == 1:
        return lambda a, b: a + b
    elif choice == 2:
        return lambda a, b: a - b
    else:
        return lambda a, b: a * b
operation = select_operation(1)    # operation = sum
print(operation(10, 6))           #16
operation = select_operation(2)    # операция = subtract
print(operation(10, 6))           #4
operation = select_operation(3)    # операция = multiply
print(operation(10, 6))
```

Преобразуване на типове

```
1)
a = 2        # число int
b = 2.5      # плаващо число float
c = a + b
print(c)
```

```
2)
a = "2"
b = 3
c = a + b
print(c)
```

```
3)
a = "2"
b = 3
c = int(a) + b
print(c)      #5
```

```
4)
a = int(15)    # a = 15
b = int(3.7)   # b = 3
c = int("4")   # c = 4
e = int(False) # e = 0
f = int(True)  # f = 1
```

```
5)
b = int("a1c")    # Error
c = int("4.7")    # Error
```

```
6)
a = "2,7"
b = 3
```



```
c = float(a) + b
print(c)          # 5.7
```

```
7)
a = float(15)      # a = 15.0
b = float(3.7)     # b = 3.7
c = float("4.7")   # c = 4.7
d = float("5")      # d = 5.0
e = float(False)   # e = 0.0
f = float(True)    # f = 1.0
```

```
8)
a = str(False)     # a = "False"
b = str(True)      # b = "True"
c = str(5)         # c = "5"
d = str(5.7)       # d = "5.7"
```

```
9) Генериране на грешки
age=22
message = "Age: " + age      # Error
print (message)
```

```
10)
age=22
message = "Age: " + str(age)  # Age: 22
print (message)
```

Забележка: Глобалният контекст предполага, че променливата е глобална, дефинирана е извън, която и да е от функциите и е достъпна за всяка функция в програмата.

Примерно решение:

```
name = "Tom"
def say_hi():
    print("Hello", name)
def say_bye():
    print("Good bye", name)
say_hi()
say_bye()
```

Забележка: За разлика от глобалните променливи, локалната променлива се дефинира вътре във функцията и е достъпна само от тази функция, т.е. има локален обхват:

Примерно решение:

```
def say_hi():
    name = "Sam"
    surname = "Johnson"
    print("Hello", name, surname)
def say_bye():
    name = "Tom"
    print("Good bye", name)
say_hi()
say_bye()
```

Забележка: Има и друга опция за дефиниране на променлива, когато локална променлива крие глобална със същото име:

Примерно решение:

```
name = "Tom"
def say_hi():
    name = "Bob"      # скриване на стойността на глобална променлива
    print("Hello", name)
def say_bye():
    print("Good bye", name)
say_hi()              # Hello Bob
say_bye()             # Good bye Tom
```

Забележка: Ако се налага промяна на глобалната променлива в локална функция, а не да се дефинира локална, тогава трябва да се използва ключова дума `global`:

Примерно решение:

```
name = "Tom"
def say_hi():
    global name
    name = "Bob"      # изменение на глобалната променлива
    print("Hello", name)
def say_bye():
    print("Good bye", name)
say_hi()              # Hello Bob
say_bye()
```

Забележка: Нелокалният израз прикрепя идентификатор към променлива от най-близкият заобикалящ контекст (с изключение на глобалния контекст). Обикновено `nonlocal` се използва във вложени функции, когато трябва да се прикачи идентификатор зад променлива или параметър на заобикалящата външна функция. Изразът демонстрира подобна ситуация:

Примерно решение:

```
def outer():          # външна функция
    n = 5
    def inner():      # вложена функция
        print(n)
    inner()           # 5
    print(n)
outer()               # 5
```

Примерно решение 2:

```
def outer():          # външна функция
    n = 5
    def inner():      # вложена функция
        n = 25
        print(n)
    inner()           # 25
    print(n)
outer()
```

Примерно решение 3:

```
def outer():          # външна функция
    n=5
    def inner():      # вложена функция
```

```
nonlocal n    # показва, че n е променлива от заобикалящата функция
n=25
print(n)
inner()      #25
print (n)
outer()
```

Задължително се изпълняват задачите от упражнение 3 и се изпращат в Team ☺

Задължителна самостоятелна работа

Задача 1. Намерете двуцифрено естествено число xy , за което се въвеждат сборът от цифрите ' $x+y$ ' и разликата между числата ' $yx - xy$ '. Да се състави програма на Elixir, която по въведени сбор и разлика извежда цифрите на числото.

Пример: Сбор $x + y = 12$; Разлика $yx - xy = 36$. Търсеното число: 48

Задача 2. Да се напише програма, която намира най-голямата и най-малката цифра на дадено неотрицателно цяло число.

Задача 3. Да се напише програма, която намира и извежда симетричното на дадено цяло число. Симетрично на дадено число се нарича число със същия знак и същите цифри, по записани в обратен ред.

Задача 4. Да се напише програма, която намира сумата на всяко трето цяло число, започвайки от 2 и ненадминавайки 100 (т.е. сумата $2+5+8+11+\dots+98$).

Задача 5. Да се намери сумата от числата в интервала от 0 до 100, които се делят на числото 3.

Задача 6. Да се напише функция **increasing_n**, която проверява дали цифрите на дадено естествено число са в нарастващ ред, четени отляво-надясно:

Пример:

за increasing (12489)
за increasing (4456)

Резултат: Да в нарастващ ред са!

Резултат: Не не са в нарастващ ред!